

FLAPPY BIRD - TUTORIAL

Kompletny przewodnik po projekcie C++ / Qt 6

Od zera do egzaminu - każda linijka kodu wyjaśniona

Projekt: Gra 2D Flappy Bird w C++/Qt 6

Przedmiot: Programowanie II - C++ obiektowy

Uczelnia: Politechnika Śląska

Autorzy: Olaf Rysiewicz, Dawid Skatula, Szymon Panek

Data: 19.06.2026

SPIS TREŚCI

1. Co to jest Qt 6 i jak działa?
2. Struktura plików projektu
3. CMakeLists.txt - jak projekt się buduje
4. main.cpp - punkt startowy programu
5. GameObject - baza dla wszystkich obiektów
6. Bird - ptak i fizyka grawitacji
7. Pipe - rury i kolizje
8. Ground - podłoga
9. PipeFactory - fabryka rur
10. GameWindow - serce całej gry
11. Pętla gry i jak Qt ją napędza
12. Rysowanie - QPainter od zera
13. Obsługa wejścia - klawiatura i mysz
14. Detekcja kolizji
15. System punktacji
16. Wzorce projektowe - OOP w praktyce
17. Zarządzanie pamięcią - new i delete
18. Pytania egzaminacyjne i odpowiedzi

1. Co to jest Qt 6 i jak dziala?

Qt (czytaj: 'kjut') to ogromna kolekcja gotowych narzedzi do budowania aplikacji w C++. Zamiast pisac wszystko od zera - obsluga okien, rysowanie pikseli, timery, zdarzenia myszy - uzywasz gotowych klas Qt.

Co Qt daje temu projektowi

- * QWidget - okno z ekranem, po ktorym mozna rysowac
- * QPainter - narzedzie do rysowania ksztaltow i tekstu
- * QTimer - licznik czasu wywolujacy funkcje co 16 ms (petla gry)
- * QKeyEvent - zdarzenia klawiatury
- * QMouseEvent - zdarzenia myszy
- * QRect - prostokat do detekcji kolizji
- * QString - string Qt (obsluguje Unicode)

Q_OBJECT i sygnaly/sloty

Qt uzywa specjalnego systemu komunikacji zwanego sygnalami i slotami. To Qt-owy odpowiednik 'zdarzen' (events). Kiedy timer 'odtyknie', wysyla sygnal timeout(). Nasz slot gameLoop() jest podlaczony do tego sygnalu i reaguje na nie.

```
// Kiedy timer 'odtyknie' (sygnal timeout), wywoluje gameLoop (slot)
connect(gameTimer, &QTimer::timeout, this, &GameWindow::gameLoop);
```

Zeby klasa mogla uzywac sygnalow i slotow, musi miec w naglowku makro Q_OBJECT:

```
class GameWindow : public QWidget {
    Q_OBJECT // to makro Qt - aktywuje caly system sygnalow/slotow
    ...
}
```

Q_OBJECT to makro - specjalny skrot, ktory Qt zamienia w dlugi kod podczas kompilacji (Meta-Object Compiler = moc). Bez niego connect() nie zadziala.

Uklad wspolrzecznych w Qt

WAZNE: W grafice komputerowej os Y jest ODWROCONA wzgledem matematyki. y=0 to GORA ekranu, y=640 to DOL ekranu. x=0 to lewa krawedz, x=480 to prawa.

```
(0,0) ----- (480,0)
|      x rosnie w PRAWO -->      |
|      y rosnie w DOL              |
|      (standardowe w grafice!)    |
(0,640) ----- (480,640)
```

2. Struktura plikow projektu

```
FlappyBird/
+-- CMakeLists.txt      <- przepis budowania (co kompilowac, jakie biblioteki)
+-- main.cpp            <- punkt startowy, funkcja main()
|
+-- GameObject.h/.cpp   <- klasa bazowa dla WSZYSTKICH obiektow gry
+-- Bird.h/.cpp         <- ptak (gracz) - fizyka, rysowanie
+-- Pipe.h/.cpp         <- rury (przeszkody) - ruch, kolizje
+-- Ground.h/.cpp       <- podloga - kolizja, rysowanie
+-- PipeFactory.h/.cpp  <- fabryka rur - tworzy losowe rury
+-- GameWindow.h/.cpp   <- glowne okno - petla gry, ekrany, wejscie
|
+-- docs/
    +-- generate_pdfs.py  <- dokumentacja projektu
    +-- generate_tutorial.py <- ten tutorial
```

Zasada: kazda klasa ma dwa pliki

Plik	Zawartosci	Po co?
.h (header)	deklaracja klasy - co potrafi, jakie ma pola	inne pliki moga #include ten naglowek
.cpp (implementacja)	cialo funkcji - jak to dziala, kod metod	kompilator tu szuka kodu

Rozdzielenie .h i .cpp to konwencja C++. Plik .h mowi 'co istnieje', .cpp mowi 'jak dziala'. Dzieki temu mozna uzyc klasy bez znajomosci implementacji.

3. CMakeLists.txt - jak projekt sie buduje

CMakeLists.txt to przepis, który mówi kompilatorowi co robić. CMake sam nie kompiluje kodu - organizuje kompilację.

Linia po linii

```
cmake_minimum_required(VERSION 3.16)
```

Qt 6 wymaga CMake w wersji co najmniej 3.16.

```
project(FlappyBird VERSION 1.0 LANGUAGES CXX)
```

Nazwa projektu: FlappyBird. Wersja: 1.0. Język: CXX = C++.

```
set(CMAKE_CXX_STANDARD 17)
```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

Używamy C++17 - wersji C++ z 2017 roku. REQUIRED ON = jeśli kompilator nie obsługuje C++17, kompilacja się nie uda.

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets)
```

Szukaj biblioteki Qt6 na komputerze. Potrzebujemy dwóch komponentów:

- * Core - podstawy (QString, QTimer, sygnały/słoty)
- * Widgets - okna i grafika (QWidget, QPainter, QKeyEvent)

```
qt_standard_project_setup()
```

Standardowe ustawienia Qt - m.in. uruchomienie narzędzia moc dla makr Q_OBJECT.

```
qt_add_executable(FlappyBird
    main.cpp
    GameObject.h   GameObject.cpp
    Bird.h         Bird.cpp
    Pipe.h         Pipe.cpp
    Ground.h       Ground.cpp
    PipeFactory.h  PipeFactory.cpp
    GameWindow.h   GameWindow.cpp
)
```

Lista WSZYSTKICH plików, które tworzą program FlappyBird.exe.

```
target_link_libraries(FlappyBird PRIVATE
    Qt6::Core
    Qt6::Widgets
)
```

Zlinkuj (połącz) nasz program z bibliotekami Qt. Bez tego kompilator nie znajdzie klas Qt.

```
set_target_properties(FlappyBird PROPERTIES
    WIN32_EXECUTABLE TRUE
)
```

Na Windowsie nie otwieraj czarnego okna konsoli - tylko okno graficzne.

4. main.cpp - punkt startowy programu

main.cpp to najkrotszy plik, ale od niego wszystko sie zaczyna. Kazdy program C++ startuje od funkcji main().

```
#include <QApplication>
#include "GameWindow.h"

int main(int argc, char* argv[])
{
    QApplication app(argc, argv);
    GameWindow window;
    window.show();
    return app.exec();
}
```

Linia po linii

#include <QApplication> - nawias ostry = plik z biblioteki Qt

#include "GameWindow.h" - cudzyslowy = nasz wlasny plik

QApplication app(argc, argv);

QApplication to SERCE kazdej aplikacji Qt. Inicjalizuje cale Qt - rejestruje aplikacje w systemie operacyjnym, ustawia obsluge zdarzen, zarzadza okienkami. Bez tego zadne okno sie nie otworzy. Przekazujemy argc i argv bo Qt moze odczytywac wlasne argumenty (np. -style).

GameWindow window;

Tworzymy obiekt GameWindow - nasze glowne okno gry. W tym momencie uruchamia sie konstruktor GameWindow::GameWindow(), ktory tworzy ptaka, podloge, fabryke rur i timer.

window.show();

Wyswietl okno. Domyslnie okna Qt sa ukryte - trzeba je pokazac.

return app.exec();

To jest PETLA ZDARZEN Qt - program tu 'wisi' i czeka na zdarzenia (klawisze, klikniecia, uplyw czasu timera). Gdy uzytkownik zamknie okno, exec() zwraca 0 i program sie konczy.

Bez app.exec() program natychmiast by sie zamknal po wyswietleniu okna! exec() utrzymuje program przy zyciu czekajac na zdarzenia.

5. GameObject - baza dla wszystkich obiektów

5.1 GameObject.h - deklaracja klasy

```
#ifndef GAMEOBJECT_H
#define GAMEOBJECT_H
#include <QPainter>

class GameObject {
public:
    GameObject();
    GameObject(float x, float y, int width, int height);
    virtual ~GameObject();

    virtual void draw(QPainter& painter) = 0; // czysto wirtualna!
    virtual void update() = 0;               // czysto wirtualna!

    float getX() const { return x; }
    float getY() const { return y; }
    int getWidth() const { return width; }
    int getHeight() const { return height; }

    void setPosition(float newX, float newY);

protected:
    float x, y;
    int width, height;
};

#endif
```

#ifndef / #define / #endif - Include Guard

Zapobiega dwukrotnemu dołączeniu tego samego nagłówka. Gdyby ten plik był dołączony dwa razy, kompilator dostałby dwie definicje tej samej klasy i zgłosił błąd. #ifndef = 'jeśli NIE zdefiniowano', #define = 'zdefiniuj', #endif = koniec warunku.

virtual ~GameObject() - wirtualny destruktor

KLUCZOWE! Wyobraź sobie:

```
GameObject* obj = new Bird(); // wskaźnik na bazę, ale obiekt to Bird
delete obj;                  // który destruktor się wywoła?
```

Bez 'virtual' - wywoła się destruktor GameObject, a destruktor Bird NIE zostanie wywołany => wyciek pamięci.
Z 'virtual' - C++ wie, żeby wywołać właściwy destruktor podklasy.

virtual void draw(...) = 0 - czysto wirtualna metoda

Oznaczenie '= 0' sprawia że:

- * GameObject jest klasą ABSTRAKCYJNA - nie można stworzyć: new GameObject() = błąd
- * Każda klasa dziedzicząca MUSI zaimplementować draw() i update()

* Gdy masz wskaźnik `GameObject*`, wywołanie `obj->draw()` automatycznie wywołuje odpowiednią implementację (ptaka, rury, podłogi) - to jest POLIMORFIZM

protected vs public vs private

Dostęp	Opis
public	dostępne dla wszystkich - z zewnątrz i w podklasach
protected	dostępne tylko w tej klasie i jej podklasach (Bird może używać x, y)
private	dostępne tylko w tej samej klasie

5.2 GameObject.cpp - implementacja

```
GameObject::GameObject()
    : x(0.0f), y(0.0f), width(0), height(0)
{}

GameObject::GameObject(float x, float y, int width, int height)
    : x(x), y(y), width(width), height(height)
{}

GameObject::~GameObject() {}

void GameObject::setPosition(float newX, float newY) {
    x = newX;
    y = newY;
}
```

Lista inicjalizacyjna (`: x(0.0f), y(0.0f)...`) to preferowany sposób inicjalizacji pól w C++. Są inicjalizowane ZANIM wykona się ciało konstruktora `{}`. W `'x(x)'` - pierwsze x to pole klasy, drugie x to parametr konstruktora.

6. Bird - ptak i fizyka grawitacji

6.1 Bird.h - naglowek

```
class Bird : public GameObject {
public:
    Bird();
    Bird(float startX, float startY);
    ~Bird() override;

    void draw(QPainter& painter) override; // override = nadpisuje metode bazowej
    void update() override;

    void jump();
    void reset();
    QRect getCollisionRect() const;

private:
    float velocityY; // aktualna predkosc pionowa
    float gravity; // przyspieszenie grawitacyjne
    float jumpForce; // sila skoku (ujemna = do gory)
    float startX, startY; // pozycja startowa (do resetu)
};
```

Slowo kluczowe 'override' (C++11) mowi kompilatorowi: ta metoda nadpisuje wirtualna metode z klasy bazowej. Jesli popelnisz literowke (np. Draw zamiast draw), kompilator zglosi blad. Bez override kompilator by po cichu stworzyl nowa metode.

6.2 Konstruktory Bird

```
Bird::Bird()
: GameObject(100.0f, 270.0f, 34, 24),
  velocityY(0.0f),
  gravity(0.5f),
  jumpForce(-9.0f),
  startX(100.0f),
  startY(270.0f)
{ }
```

Wywołuje konstruktor klasy bazowej GameObject(100, 270, 34, 24) - ptak zaczyna na pozycji (100, 270), ma wymiary 34x24 piksele.

Dlaczego jumpForce = -9.0f? W grafice y=0 jest na GORZE ekranu. Zeby leciec w gore, musisz ZMNIEJSZAC y, czyli predkosc musi byc UJEMNA.

6.3 Fizyka grawitacji - Bird::update()

```
void Bird::update()
{
    velocityY += gravity; // predkosc rosnie o 0.5 kazda klatke
    if (velocityY > 14.0f) velocityY = 14.0f; // max predkosc spadania
    y += velocityY; // zmien pozycje o predkosc
```



```
}
```

Tabela - symulacja fizyki klatka po klatce

Klatka	velocityY przed	po += gravity	y zmienia sie o	Kierunek
1 (skok)	-9.0	-8.5	-8.5	GORA
5	-7.0	-6.5	-6.5	GORA
10	-4.5	-4.0	-4.0	GORA
19	+0.5	+1.0	+1.0	DOL
28	+5.0	+5.5	+5.5	DOL
35+	+14.0	+14.0	+14.0	DOL (max)

Ograniczenie velocityY > 14.0f to 'terminal velocity' (predkosc koncowa). Zapobiega nierealistycznie szybkiemu spadaniu.

6.4 Skok - Bird::jump()

```
void Bird::jump()
{
    velocityY = jumpForce; // = -9.0f, resetuje predkosc na ujemna
}
```

Skok nie DODAJE predkosci - USTAWIA ja na -9. Dzieki temu wielokrotne klikniecie nie pozwala leciec w nieskonczonosc.

6.5 Prostokat kolizji - getCollisionRect()

```
QRect Bird::getCollisionRect() const
{
    const int margin = 5;
    return QRect(
        static_cast<int>(x) + margin,
        static_cast<int>(y) + margin,
        width - 2 * margin,
        height - 2 * margin
    );
}
```

Kolizja sprawdzana jest na prostokacie MNIEJSZYM o 5 pikseli ze wszystkich stron. Dlaczego? Bo rysowanie ptaka (elipsa z dziobem) wychodzi poza prostokata. Gdybysmy uzywali pelnych wymiarow, gra bylaby niesprawiedliwa.

6.6 Rysowanie ptaka - Bird::draw()

```
void Bird::draw(QPainter& painter) {
    int bx = static_cast<int>(x); // float -> int dla rysowania
    int by = static_cast<int>(y);

    // Cialo (zolta elipsa)
    painter.setBrush(QColor(255, 220, 0));
}
```

```

painter.setPen(QPen(QColor(180, 140, 0), 2));
painter.drawEllipse(bx, by, width, height);

// Skrzydło (ciemniejszy oval)
painter.setBrush(QColor(220, 170, 0));
painter.setPen(Qt::NoPen);
painter.drawEllipse(bx + 4, by + height / 2, 14, 8);

// Oko (białe + czarna zrenica)
painter.setBrush(Qt::white);
painter.drawEllipse(bx + width/2 + 2, by + 4, 11, 11);
painter.setBrush(Qt::black);
painter.drawEllipse(bx + width/2 + 5, by + 7, 5, 5);

// Dziób (pomarańczowy trójkąt - QPolygon)
QPolygon beak;
beak << QPoint(bx + width,      by + height/2 - 3)
      << QPoint(bx + width + 10, by + height/2)
      << QPoint(bx + width,      by + height/2 + 3);
painter.setBrush(QColor(255, 140, 0));
painter.drawPolygon(beak);
}

```

setBrush = kolor wypełnienia, setPen = kolor/grubość konturu, Qt::NoPen = brak konturu.

QPolygon = wielokąt z punktów. Trzy punkty tworzą trójkąt (dziób). << to operator Qt do dodawania punktów do wielokąta.

7. Pipe - rury i kolizje

7.1 Pipe.h - stała PIPE_WIDTH

```
static constexpr int PIPE_WIDTH = 58;
```

static = należy do klasy, nie do obiektu (jeden egzemplarz dla wszystkich rur). constexpr = wartość znana w czasie kompilacji, nie może się zmienić. Dostęp: Pipe::PIPE_WIDTH.

Dlaczego static constexpr zamiast #define? Bo ma typ int i należy do przestrzeni nazw klasy. #define jest globalny i może kolidować z innymi nazwami.

7.2 Konstruktor - walidacja i wyjatki

```
Pipe::Pipe(float x, int gapCenterY, int gapHeight, int screenHeight)
    : GameObject(x, 0.0f, PIPE_WIDTH, screenHeight), ...
{
    // Walidacja: minimalna przerwa
    if (gapHeight < 90) {
        throw std::invalid_argument(
            "Przerwa między rurami jest za mała! Minimum to 90 pikseli."
        );
    }

    // Walidacja: min odległość od góry
    if (gapCenterY - gapHeight / 2 < 40) {
        throw std::out_of_range(
            "Środek przerwy jest za blisko górnej krawędzi ekranu."
        );
    }

    // Walidacja: min odległość od dołu
    if (gapCenterY + gapHeight / 2 > screenHeight - 100) {
        throw std::out_of_range(
            "Środek przerwy jest za blisko dolnej krawędzi ekranu."
        );
    }
}
```

Rura zaczyna na y=0 (góra ekranu) i ma wysokość całego ekranu - rysuje zarówno górną jak i dolną rurę.

throw rzuca wyjątek - sygnalizuje że coś poszło nie tak. std::invalid_argument = nieprawidłowy argument.

std::out_of_range = wartość poza dopuszczalnym zakresem.

7.3 Schemat rysowania rury

```
y=0
+-----+ <- gorna rura (cialo)
|         |
+-----+ <- topEnd = gapCenterY - gapHeight/2
+-----+ <- kapitel (szerszy o 6px z kazdej strony)

[ przerwa ] <- ptak tu leci
```

```

+-----+ <- kapitel dolnej rury
+-----+ <- bottomStart = gapCenterY + gapHeight/2
|         |
+-----+ <- screenHeight (y=640)
y=640

```

7.4 Ruch i sprawdzenie czy poza ekranem

```

void Pipe::update() {
    x -= speed; // przesun w lewo o 'speed' pikseli (domyslnie 3.0)
}

bool Pipe::isOffScreen() const {
    return (x + PIPE_WIDTH + 10) < 0; // prawa krawedz za lewa krawedzia ekranu
}

```

7.5 Prostokaty kolizji - dwa oddzielne

```

QRect Pipe::getTopRect() const {
    int topEnd = gapCenterY - gapHeight / 2;
    return QRect(static_cast<int>(x), 0, PIPE_WIDTH, topEnd);
}

QRect Pipe::getBottomRect() const {
    int bottomStart = gapCenterY + gapHeight / 2;
    int bottomH = screenHeight - bottomStart;
    return QRect(static_cast<int>(x), bottomStart, PIPE_WIDTH, bottomH);
}

```

Dwa oddzielne prostokaty - top od y=0 do topEnd, bottom od bottomStart do dolu ekranu.

8. Ground - podloga

8.1 Konstruktor

```
Ground::Ground()
    : GameObject(0.0f, 560.0f, 480, 80), // y=640-80=560
      screenWidth(480)
{}

```

Podloga zaczyna na y=560 (640 - 80 = 560), ma szerokosc 480px i wysokosc 80px. Wyrownana do dolnej krawedzi ekranu.

8.2 Rysowanie

```
void Ground::draw(QPainter& painter) {
    int gy = static_cast<int>(y); // y=560

    painter.setBrush(QColor(80, 170, 70)); // zielona trawa
    painter.drawRect(0, gy, screenWidth, 16); // 16px trawa

    painter.setBrush(QColor(60, 140, 50)); // ciemniejszy pasek
    painter.drawRect(0, gy + 14, screenWidth, 4); // 4px separator

    painter.setBrush(QColor(210, 170, 110)); // piasek/ziemia
    painter.drawRect(0, gy + 18, screenWidth, height - 18); // reszta

    painter.setBrush(QColor(180, 140, 80)); // ciemniejsza linia w ziemi
    painter.drawRect(0, gy + 30, screenWidth, 8);
}

```

Podloga to 4 prostokata tworzące warstwowy efekt: trawa -> separator -> piasek -> podpowierzchniowa warstwa.

8.3 Pusta metoda update()

```
void Ground::update()
{
    // Static - no movement
}

```

Podloga sie nie rusza - override MUSI istniec (klasa bazowa ma = 0), ale cialo jest puste. To jest legalne C++.

9. PipeFactory - fabryka rur

9.1 Wzorzec Factory Method

PipeFactory implementuje wzorzec Factory Method. Zamiast tworzyć rury bezpośrednio (`new Pipe(...)`) w `GameWindow`, delegujemy to do fabryki. Korzystając:

- * Logika losowania pozycji jest w jednym miejscu
- * Można łatwo zmienić trudność
- * `GameWindow` nie musi znać szczegółów tworzenia rur

9.2 Konstruktor i inicjalizacja losowania

```
PipeFactory::PipeFactory(int screenWidth, int screenHeight, int groundHeight)
: screenWidth(screenWidth), screenHeight(screenHeight),
  groundHeight(groundHeight),
  gapHeight(165),          // domyślna wysokość przerwy
  pipeSpeed(3.0f)          // domyślna szybkość
{
    srand(static_cast<unsigned int>(time(nullptr)));
}
```

`srand(time(nullptr))` - inicjalizacja generatora liczb pseudolosowych. `time(nullptr)` zwraca czas w sekundach od 1970-01-01 (Unix timestamp). Dzięki temu każde uruchomienie daje inne rury.

Bez `srand()`, `rand()` zawsze daje tę samą sekwencję liczb - rury byłyby zawsze w tych samych miejscach!

9.3 createPipe - losowanie pozycji

```
Pipe* PipeFactory::createPipe()
{
    int minGapCenter = 60 + gapHeight / 2;
    int maxGapCenter = (screenHeight - groundHeight) - 60 - gapHeight / 2;

    // Losuj środek przerwy w bezpiecznym zakresie
    int range = maxGapCenter - minGapCenter;
    int gapCenter = minGapCenter + (range > 0 ? rand() % range : 0);

    try {
        Pipe* pipe = new Pipe(
            static_cast<float>(screenWidth + 10), // start za prawym brzegiem
            gapCenter,
            gapHeight,
            screenHeight
        );
        pipe->setSpeed(pipeSpeed);
        return pipe;
    }
    catch (const std::invalid_argument& e) {
        // Jeśli walidacja nie przeszła, stwórz bezpieczną rurę
        Pipe* safePipe = new Pipe(
            static_cast<float>(screenWidth + 10),
```

```

        screenHeight / 2, // srodek ekranu
        170,              // bezpieczna przerwa
        screenHeight
    );
    safePipe->setSpeed(pipeSpeed);
    return safePipe;
}

catch (const std::out_of_range& e) {
    // Podobnie - fallback
    ...
}
}

```

rand() % range - operator modulo. Losuje liczbe z zakresu [0, range-1]. catch lapie wyjatki rzucone przez konstruktor Pipe. Dzieki temu fabryka NIGDY nie zwraca nullptr.

9.4 setDifficulty - poziomy trudnosc

```

void PipeFactory::setDifficulty(int level)
{
    if (level < 1) level = 1;
    if (level > 5) level = 5;

    gapHeight = 165 - (level - 1) * 15;    // 165, 150, 135, 120, 105
    pipeSpeed = 3.0f + (level - 1) * 0.5f; // 3.0, 3.5, 4.0, 4.5, 5.0

    if (gapHeight < 100) gapHeight = 100; // minimalny prog bezpieczenstwa
}

```

Poziom	Przerwa (px)	Predkosc (px/kl.)
1	165	3.0
2	150	3.5
3	135	4.0
4	120	4.5
5	105	5.0

10. GameWindow - serce calej gry

10.1 Stany gry - enum class GameState

```
enum class GameState {  
    START,      // ekran startowy - gra jeszcze nie zaczeta  
    PLAYING,    // gra w toku  
    GAMEOVER    // ptak umarl  
};
```

enum class (C++11) to silnie typowany enum. Nie mozna przypadkowo porownac GameState z liczba. GameState::START nie jest rowne 0 tak po prostu - musisz byc jawny. Zamiast wielu flag (isStarted, isDead...) jeden GameState.

10.2 Stale rozmiaru i pola klasy

```
static const int SCREEN_WIDTH  = 480;  
static const int SCREEN_HEIGHT = 640;  
static const int GROUND_HEIGHT = 80;  
  
Bird*      bird;          // wskaznik na obiekt ptaka  
Ground*    ground;        // wskaznik na podloge  
PipeFactory* pipeFactory; // wskaznik na fabryke rur  
std::vector<Pipe*> pipes;  // dynamiczna lista wskaznikow na rury  
  
QTimer* gameTimer;        // timer wywolujacy gameLoop co 16ms  
  
int frameCount;           // licznik klatek  
int pipeSpawnEvery;       // co ile klatek spawnowac rure (=88)  
int score;                // aktualny wynik  
int bestScore;            // rekord sesji  
GameState gameState;      // aktualny stan gry
```

10.3 Konstruktor GameWindow

```
GameWindow::GameWindow(QWidget* parent)  
    : QWidget(parent),  
      bird(nullptr), ground(nullptr), pipeFactory(nullptr),  
      gameTimer(nullptr),  
      frameCount(0), pipeSpawnEvery(88),  
      score(0), bestScore(0),  
      gameState(GameState::START)  
{  
    setFixedSize(SCREEN_WIDTH, SCREEN_HEIGHT); // staly rozmiar 480x640  
    setWindowTitle("Flappy Bird - Projekt C++ / Qt 6");  
    setFocusPolicy(Qt::StrongFocus); // odbieraj klawisze  
  
    bird      = new Bird(100.0f, static_cast<float>(SCREEN_HEIGHT/2 - 60));  
    ground    = new Ground(SCREEN_WIDTH, SCREEN_HEIGHT, GROUND_HEIGHT);  
    pipeFactory = new PipeFactory(SCREEN_WIDTH, SCREEN_HEIGHT, GROUND_HEIGHT);
```



```
gameTimer = new QTimer(this); // 'this' = rodzic; Qt usunie timer auto.  
connect(gameTimer, &QTimer::timeout, this, &GameWindow::gameLoop);  
gameTimer->start(16); // co 16ms = ~62.5 FPS  
}
```

Wskazniki inicjalizowane na nullptr - dobra praktyka. new Bird/Ground/PipeFactory tworzy obiekty na STERCIE (heap) - muszą żyć przez cały czas działania okna.

gameTimer = new QTimer(this) - 'this' to RODZIC timera. Qt automatycznie usunie timera gdy GameWindow zostanie zniszczony.

10.4 Destruktor GameWindow

```
GameWindow::~~GameWindow()  
{  
    delete bird;  
    delete ground;  
    delete pipeFactory;  
    // gameTimer - NIE delete, Qt robi to przez system rodzic-dziecko  
  
    for (Pipe* pipe : pipes) {  
        delete pipe; // usun OBIEKT Pipe  
    }  
    pipes.clear(); // usun WSKAZNIKI z wektora  
}
```

Dlaczego dwa kroki dla rur? std::vector<Pipe> przechowuje wskazniki (adresy). pipes.clear() usuwa adresy z wektora, ale nie usuwa obiektów Pipe pod tymi adresami. Najpierw delete pipe (usun obiekt), potem pipes.clear() (wyczyszc wektor).*

11. Petla gry i jak Qt ja napędza

11.1 Schemat przepływu

```
QTimer (co 16ms)
| sygnal: timeout
v
GameWindow::gameLoop()
+-- bird->update()          <- fizyka ptaka
+-- spawn rur (co 88 klatek) <- nowa rura z fabryki
+-- pipes[i]->update()      <- ruch rur w lewo
+-- usun rury za ekranem    <- delete + erase
+-- checkCollisions()       <- czy ptak zyje?
+-- updateScore()           <- punkty
+-- update()                <- prosba o przerysowanie
    |
    v
paintEvent()                <- Qt wywola, rysuje klatke
```

11.2 Kod gameLoop()

```
void GameWindow::gameLoop()
{
    if (gameState != GameState::PLAYING) {
        update(); // odswiezaj ekran nawet na menu/game over
        return;
    }

    frameCount++;

    bird->update();

    // Co 88 klatek (~1.4 sek) spawnuj nowa rure
    if (frameCount % pipeSpawnEvery == 0) {
        try {
            Pipe* newPipe = pipeFactory->createPipe();
            pipes.push_back(newPipe);
        }
        catch (const std::exception& e) { }
    }

    // Iteracja WSTECZNA dla bezpiecznego usuwania
    for (int i = static_cast<int>(pipes.size()) - 1; i >= 0; --i) {
        pipes[i]->update();

        if (pipes[i]->isOffScreen()) {
            delete pipes[i]; // usun obiekt
            pipes.erase(pipes.begin() + i); // usun wskaznik z wektora
        }
    }

    checkCollisions();
}
```

```

if (gameState == GameState::PLAYING) {
    updateScore();
}

update(); // prosba o przerysowanie okna
}

```

Dlaczego iteracja WSTECZNA gdy usuwamy rury?

erase() zmienia indeksy! Jesli usuniesz element o indeksie 2, element o indeksie 3 staje sie nowym indeksem 2. Przy iteracji do przodu po usunieciu elementu pominiesz nastepny. Iterujac wstecz, usuniecie indeksu i nie wpływa na indeksy 0..i-1 ktore jeszcze nie byly sprawdzone.

frameCount % pipeSpawnEvery == 0 - operator modulo

% to operator reszty z dzielenia. Gdy frameCount = 88, $88 \% 88 = 0$ -> spawn. Gdy frameCount = 176, $176 \% 88 = 0$ -> spawn. Co 88 klatek przy 62.5 FPS = co ~1.4 sekundy.

update() vs paintEvent()

update() w Qt PROSI system o przerysowanie okna. Nie rysuje bezpośrednio - Qt wywola paintEvent() asynchronicznie (gdy system jest gotowy). Nie mozna wywolac paintEvent bezpośrednio.

11.3 Kolejnosć rysowania w paintEvent()

```

void GameWindow::paintEvent(QPaintEvent*)
{
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing);

    drawBackground(painter); // 1. tlo (niebo, chmury)

    for (Pipe* pipe : pipes) { // 2. rury (za podloga i ptakiem)
        pipe->draw(painter);
    }

    ground->draw(painter); // 3. podloga (przykrywa dol rur)
    bird->draw(painter); // 4. ptak (na wierzchu)
    drawUI(painter); // 5. wynik (zawsze na wierzchu)

    if (gameState == GameState::START)
        drawStartScreen(painter); // 6. nakladka menu
    else if (gameState == GameState::GAMEOVER)
        drawGameOverScreen(painter);
}

```

Kolejnosć jest krytyczna - kazda nastepna warstwa rysowana jest NA poprzedniej. Antialiasing = wygładzanie krawedzi (bez 'schodkow' na elipsach i liniach).

12. Rysowanie - QPainter od zera

12.1 Podstawowe metody QPainter

Metoda	Co robi
setBrush(kolor)	ustaw kolor wypełnienia kształtów
setPen(kolor/QPen)	ustaw kolor i grubosc konturu
setPen(Qt::NoPen)	brak konturu (same wypełnienie)
drawRect(x,y,w,h)	rysuj prostokąt o lewym górnym rogu (x,y)
drawEllipse(x,y,w,h)	rysuj elipse wpisana w prostokąt (x,y,w,h)
drawPolygon(QPolygon)	rysuj wielokąt z listy punktów
drawText(QRect, flagi, tekst)	rysuj tekst w prostokacie z wyrównaniem
setFont(QFont)	ustaw czcionke, rozmiar, styl (Bold/Italic)
fillRect(QRect, pedziel)	szybkie wypełnienie prostokatu gradientem/kolorem

12.2 QColor - kolory

```
QColor(255, 220, 0)           // RGB - zolty (ptak)
QColor(78, 175, 65)           // zielony (rury)
QColor(255, 255, 255, 200)    // biały z alpha=200 (polprzezroczysta chmura)
Qt::white                     // predefiniowany biały
Qt::black                     // predefiniowany czarny
```

Czwarty parametr QColor to alpha (przezroczystosc): 0 = pelna przezroczystosc, 255 = pelne wypełnienie.

12.3 Gradient nieba

```
QLinearGradient skyGrad(0, 0, 0, SCREEN_HEIGHT - GROUND_HEIGHT);
skyGrad.setColorAt(0.0, QColor(100, 180, 210)); // niebieski na gorze
skyGrad.setColorAt(1.0, QColor(175, 225, 240)); // jasniejszy na dole
painter.fillRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, skyGrad);
```

QLinearGradient(x1,y1,x2,y2) - gradient liniowy od (0,0) do (0,560) (pionowy). setColorAt(0.0,...) = kolor na poczatku, setColorAt(1.0,...) = kolor na koncu.

12.4 Efekt cienia tekstu (drop shadow)

```
// Cien (przesuniety o 2px)
painter.setPen(QColor(0, 0, 0, 160));
painter.drawText(scoreArea.adjusted(2, 2, 2, 2), Qt::AlignHCenter, scoreStr);

// Biały tekst
painter.setPen(Qt::white);
painter.drawText(scoreArea, Qt::AlignHCenter, scoreStr);
```

Technika drop shadow: rysuj tekst dwa razy. Raz ciemny przesuniety o (2,2), raz biały w normalnej pozycji. Efekt cienia bez specjalnych shaderow.

13. Obsługa wejścia - klawiatura i mysz

13.1 keyPressEvent

```
void GameWindow::keyPressEvent(QKeyEvent* event)
{
    if (event->key() == Qt::Key_Space || event->key() == Qt::Key_Up) {
        switch (gameState) {
            case GameState::START:
                startGame();
                break;
            case GameState::PLAYING:
                bird->jump();
                break;
            case GameState::GAMEOVER:
                resetGame();
                break;
        }
    }

    if (event->key() == Qt::Key_Escape) {
        close();
    }
}
```

switch na gameState - ta sama akcja (spacja/strzałka góra) robi co innego zależnie od stanu gry. To implementacja AUTOMATU STANOW (State Machine).

13.2 mousePressEvent

```
void GameWindow::mousePressEvent(QMouseEvent* event)
{
    if (event->button() == Qt::LeftButton) {
        switch (gameState) {
            // identyczny switch jak wyżej
        }
    }
}
```

Identyczna logika jak klawiatura - lewy przycisk myszy robi to samo co spacja. Sprawdzamy event->button() == Qt::LeftButton żeby ignorować prawy i środkowy przycisk.

13.3 startGame vs resetGame

```
void GameWindow::startGame()
{
    gameState = GameState::PLAYING;
    frameCount = 0;
    score = 0;
    bird->reset();
    bird->jump(); // od razu skok żeby nie spasc
    for (Pipe* p : pipes) delete p;
}
```

```

    pipes.clear();
}

void GameWindow::resetGame()
{
    for (Pipe* p : pipes) delete p;
    pipes.clear();
    bird->reset();
    frameCount = 0;
    score      = 0;
    gameState = GameState::START; // wróc do menu, nie PLAYING
}

```

startGame -> stan PLAYING. resetGame -> stan START (menu). Ptak od razu skacze w startGame bo inaczej spadł zanim gracz się zorientuje.

13.4 Diagram przejść między stanami



14. Detekcja kolizji

14.1 Kod checkCollisions()

```
void GameWindow::checkCollisions()
{
    QRect birdRect = bird->getCollisionRect(); // zmniejszony prostokąt ptaka

    // 1. Kolizja z podloga
    if (birdRect.intersects(ground->getCollisionRect())) {
        gameState = GameState::GAMEOVER;
        if (score > bestScore) bestScore = score;
        return;
    }

    // 2. Kolizja z gorna krawedzia ekranu
    if (bird->getY() < -bird->getHeight()) {
        gameState = GameState::GAMEOVER;
        if (score > bestScore) bestScore = score;
        return;
    }

    // 3. Kolizja z rurami
    for (Pipe* pipe : pipes) {
        if (pipe == nullptr) continue;

        if (birdRect.intersects(pipe->getTopRect()) ||
            birdRect.intersects(pipe->getBottomRect())) {
            gameState = GameState::GAMEOVER;
            if (score > bestScore) bestScore = score;
            return;
        }
    }
}
```

QRect::intersects(QRect)

Zwraca true jeśli prostokąty mają wspólny obszar (nakładają się). Jest to podstawowa technika AABB (Axis-Aligned Bounding Box) - najszybsza metoda detekcji kolizji w grach 2D.

Kolejność sprawdzeń nie jest przypadkowa: podłoga jest sprawdzana pierwsza (najczęstszy przypadek), potem granica, potem rury.

return po znalezieniu kolizji - nie sprawdzamy dalej. Wystarczy jedna kolizja żeby gra się skończyła. || to operator logiczny OR - wystarczy kolizja z GÓRNA LUB DOLNA rura.

Dlaczego zmniejszony hitbox ptaka?

Grafika ptaka (elipsa + dziób wystaje) jest większa niż prostokąt kolizji. Margines 5px ze wszystkich stron daje graczowi trochę taryfy ulgzonej - gra wygląda sprawiedliwie, ptak nie 'ginie' zanim wizualnie dotknie rury.

15. System punktacji

15.1 Kod updateScore()

```
void GameWindow::updateScore()
{
    float birdCenterX = bird->getX() + bird->getWidth() / 2.0f;

    for (Pipe* pipe : pipes) {
        if (pipe == nullptr || pipe->isPassed()) continue;

        float pipeCenterX = pipe->getX() + Pipe::PIPE_WIDTH / 2.0f;

        if (birdCenterX > pipeCenterX) {
            pipe->setPassed(true);
            score++;
        }
    }
}
```

Jak działa krok po kroku

- * 1. Obliczamy srodek X ptaka (pozycja + polowa szerokosci)
- * 2. Dla kazdej rury ktora JESZCZE NIE JEST ZALICZONA (!isPassed())
- * 3. Obliczamy srodek X rury
- * 4. Jesli srodek ptaka jest ZA srodkiem rury -> punkt!
- * 5. Oznaczamy rure jako passed = true zeby nie liczyc jej ponownie

Dlaczego srodek rury, a nie jej lewa krawedz? Chcemy przyznac punkt gdy ptak jest w polowie drogi przez rure - brzmi bardziej naturalnie i sprawiedliwie.

Flaga 'passed' w klasie Pipe jest konieczna - bez niej ptak zdobylby punkt w KAZDEJ klatce gdy jest za rura (co 16ms = 62 punkty na sekunde!).

16. Wzorce projektowe - OOP w praktyce

16.1 Polimorfizm

Polimorfizm (wielopostaciowos) - ten sam interfejs, rozna implementacja. Dzieki 'virtual' w klasie bazowej, wywołanie `->draw()` na wskaźniku `GameObject*` automatycznie trafia do właściwej implementacji.

```
// Kompilator NIE WIE w czasie kompilacji który draw() wywola
// Decyzja jest podejmowana w CZASIE WYKONANIA (dynamic dispatch)
GameObject* obj = new Bird();
obj->draw(painter); // -> wywołuje Bird::draw(), nie GameObject::draw()
```

Mechanizm: każda klasa z virtual metoda ma ukryta tablice wskaźników (vtable). Wywołanie przez wskaźnik bazowy sprawdza vtable i skacze do właściwej funkcji.

16.2 Dziedziczenie i hierarchia klas

```
GameObject (abstrakcyjna - baza)
+-- Bird      : public GameObject
+-- Pipe      : public GameObject
+-- Ground    : public GameObject

QWidget (Qt)
+-- GameWindow : public QWidget
```

Bird, Pipe i Ground dziedziczą wszystkie pola z GameObject (x, y, width, height) i metody (getX, getY, setPosition) bez potrzeby ich powtarzania. Muszą za to zaimplementować draw() i update().

16.3 Wzorzec Factory Method (PipeFactory)

```
// Zamiast:
Pipe* pipe = new Pipe(480+10, rand()%300 + 150, 165, 640); // logika rozrzucona

// Używamy:
Pipe* pipe = pipeFactory->createPipe(); // enkapsulacja logiki w fabryce
```

GameWindow nie musi wiedzieć JAK rura jest tworzona - tylko prosi fabrykę.

16.4 Automat stanów (State Machine)

GameState (START/PLAYING/GAMEOVER) to prosty automat stanów. Zamiast wielu flag (isStarted, isDead, isPlaying...) jeden enum. Logika jest czysta - switch(gameState) zamiast gąszczu if/else.

16.5 Enkapsulacja

Pola klas są private lub protected - nie można ich zmienić z zewnątrz bez przejścia przez metody klasy. Np. pozycja ptaka zmienia się tylko przez update() i reset() - nie można przypadkowo ustawić y = -1000.

17. Zarządzanie pamięcią - new i delete

17.1 Sterta (heap) vs Stos (stack)

```
// STOS - automatycznie zwolnione gdy wyjdiesz z zakresu {}
{
    Bird bird;    // powstaje na stosie
} // <-- tutaj bird jest automatycznie niszczone

// STERTA - żyje dopóki nie wywołasz delete
Bird* bird = new Bird(); // powstaje na sterpie, zwraca wskaźnik
// ... bird żyje przez cały czas ...
delete bird;           // teraz jest niszczone
```

Obiekty gry (bird, ground, pipes) muszą żyć przez cały czas działania GameWindow, więc są na sterpie. Lokalne zmienne na stosie znikają po wyjściu z konstruktora.

17.2 Reguła: każde new ma delete

Gdzie new	Gdzie delete
GameWindow() -> new Bird	~GameWindow() -> delete bird
GameWindow() -> new Ground	~GameWindow() -> delete ground
GameWindow() -> new PipeFactory	~GameWindow() -> delete pipeFactory
GameWindow() -> new QTimer(this)	automatycznie przez Qt (parent=this)
gameLoop -> pipeFactory->createPipe()	gameLoop -> delete pipe (gdy offscreen) + ~GameWindow

17.3 Wyciek pamięci (memory leak)

Wyciek pamięci = new bez delete. Program zużywa coraz więcej RAM, aż system zabije proces. W tej grze: gdyby nie usuwać rur wychodzących za ekran, każde uruchomienie by tworzyło setki rur w pamięci.

17.4 Bezpieczne usuwanie z wektora

```
// NIEBEZPIECZNE - iteracja w przód
for (int i = 0; i < pipes.size(); i++) {
    if (pipes[i]->isOffScreen()) {
        delete pipes[i];
        pipes.erase(pipes.begin() + i); // przesuwamy indeksy!
        // pipes[i+1] staje się pipes[i] - pomijamy go!
    }
}

// BEZPIECZNE - iteracja wsteczna
for (int i = static_cast<int>(pipes.size()) - 1; i >= 0; --i) {
    if (pipes[i]->isOffScreen()) {
        delete pipes[i];           // usun obiekt
        pipes.erase(pipes.begin() + i); // usun wskaźnik - bezpieczne!
    }
}
```

}

18. Pytania egzaminacyjne i odpowiedzi

P1: Co to jest klasa abstrakcyjna i dlaczego GameObject nie jest?

O: Klasa abstrakcyjna to klasa z co najmniej jedną czystą metodą wirtualną (= 0). GameObject ma draw() i update() jako czyste - nie można stworzyć obiektu new GameObject() bezpośrednio. Wymusza to, żeby każda podklasa (Bird, Pipe, Ground) miała własną implementację rysowania i aktualizacji.

P2: Co to jest virtual destruktor i dlaczego jest konieczny?

O: Bez virtual ~GameObject(), jeżeli usuniesz obiekt Bird przez wskaźnik GameObject*, wywoła się tylko destruktor GameObject - destruktor Bird nie zostanie wywołany. To prowadzi do wycieków pamięci. virtual sprawia, że C++ wywołuje destruktor właściwej podklasy.

P3: Jak działa Q_OBJECT i po co jest?

O: Q_OBJECT to makro Qt, które musi być w każdej klasie używając sygnałów i slotów. Kompilator Qt (moc = Meta-Object Compiler) przetwarza je i generuje kod obsługujący system sygnałów/slotów, introspekcję i inne mechanizmy Qt. Bez niego connect() nie zadziała.

P4: Dlaczego petla gry korzysta z timera, a nie z petli while(true)?

O: Qt jest frameworkiem zdarzeniowym - ma własną petlę zdarzeń (app.exec()). Blokująca petla while(true) zablokowałaby te petle, uniemożliwiając obsługę klawiszy, myszy i rysowania. QTimer współpracuje z petlą zdarzeń - co 16ms dodaje zdarzenie timeout, które jest obsługiwane gdy petla jest gotowa.

P5: Dlaczego iterujesz wstecznie gdy usuwasz rury?

O: std::vector::erase przesuwa wszystkie elementy za usuniętym miejscem o jeden. Przy iteracji do przodu po usunięciu elementu i, element i+1 staje się nowym i - więc go pominiesz. Iteracja wsteczna eliminuje ten problem: usunięcie i nie wpływa na elementy 0..i-1.

P6: Co to jest Wzorzec Fabryki i dlaczego go używamy?

O: Factory Method to wzorzec kreacyjny - zamiast tworzyć obiekty bezpośrednio, mamy dedykowaną klasę (fabrykę) która to robi. PipeFactory::createPipe() enkapsuluje losowanie pozycji, walidację i obsługę wyjątków. GameWindow nie musi znać szczegółów - po prostu prosi fabrykę o nową rurę.

P7: Jak działa detekcja kolizji?

O: Używamy QRect::intersects(). Każdy obiekt może zwrócić swój prostokąt kolizji. Ptak ma zmniejszony prostokąt (margines 5px). intersects() zwraca true jeśli prostokąty mają wspólny obszar. Sprawdzamy: ptak-podłoga, ptak-górna granica, ptak-górna część rury, ptak-dolna część rury.

P8: Dlaczego pozycja ptaka jest float, a nie int?

O: Fizyka grawitacji (prędkość 0.5f/klatkę) wymaga ułamkowej precyzji. Gdyby y było int, ptak poruszałby się skokami po 1 piksel i animacja byłaby szarpana. float pozwala na płynny ruch, a konwersja do int następuje

tylko przy rysowaniu (`static_cast<int>(y)`).

P9: Co robi `srand(time(nullptr))` w `PipeFactory`?

O: `rand()` generuje pseudolosowe liczby - zawsze ta sama sekwencja. `srand()` ustawia ziarno (seed) - punkt startowy tej sekwencji. `time(nullptr)` zwraca aktualny czas Unix (sekundy od 1.1.1970). Ponieważ czas zmienia się co sekundę, każde uruchomienie programu daje inną sekwencję losową - czyli inne pozycje rur.

P10: Jaka jest różnica między `START` a `PLAYING` a `GAMEOVER`?

O: `START` = menu startowe, gra nie zaczęta, brak rur. `PLAYING` = gra w toku, petla gry aktywna, rury się pojawiają, fizyka działa. `GAMEOVER` = ptak umarł, ekran końcowy. Wejście przekłącza: `START->PLAYING` (`startGame`), `PLAYING->GAMEOVER` (kolizja), `GAMEOVER->START` (`resetGame`).

P11: Dlaczego `Pipe::PIPE_WIDTH` jest `static constexpr`?

O: `static` = jedna kopia dla całej klasy (nie per-obiekt). `constexpr` = wartość znana w czasie kompilacji, może być użyta w wyrażeniach stałych. Szerokość rury jest stała dla wszystkich rur, więc nie ma sensu przechowywać jej w każdym obiekcie. Dostępna jako `Pipe::PIPE_WIDTH`.

P12: Co to jest `enum class` i czym różni się od zwykłego `enum`?

O: `enum class` (C++11) to silnie typowany `enum`. Zwykły `enum` zanieczyszcza przestrzeń nazw - `START` byłoby dostępne globalnie i mogłoby kolidować z innymi nazwami. `enum class` wymaga pełnej kwalifikacji: `GameState::START`. Dodatkowo nie konwertuje się automatycznie do `int`.

P13: Co to jest lista inicjalizacyjna w konstruktorze i po co?

O: Lista inicjalizacyjna to część po dwukropku przed ciałem konstruktora: `Bird::Bird() : velocityY(0.0f), gravity(0.5f) { }`. Inicjalizuje pola PRZED wejściem do ciała konstruktora. Jest wydajniejsza niż przypisanie w `{ }` bo unika podwójnej inicjalizacji. Dla pól `const` i referencji jest WYMAGANA.

P14: Co się dzieje gdy wywołujesz `update()` w `gameLoop`?

O: `update()` w Qt nie rysuje bezpośrednio - to prośba do systemu o przerysowanie okna. Qt kolejkuje te prośby i gdy jest gotowy, wywołuje `paintEvent()`. Nie można wywołać `paintEvent()` bezpośrednio - to błąd projektowy.

Podsumowanie - co pokazuje ten projekt

Elementy OOP

Koncept	Gdzie w projekcie
Dziedziczenie	Bird, Pipe, Ground : public GameObject
Polimorfizm	virtual draw/update, wywołanie przez wskaźnik bazowy
Enkapsulacja	private/protected pola, publiczne metody dostępne
Klasa abstrakcyjna	GameObject z = 0 metodami
Wirtualny destruktor	~GameObject() virtual
Konstruktory	domyslny i parametryczny w kazdej klasie
Wyjatk	throw w Pipe::Pipe(), catch w PipeFactory
Wzorzec Factory	PipeFactory::createPipe()
Automat stanow	enum class GameState {START, PLAYING, GAMEOVER}

Elementy C++ i Qt

Technika	Opis
static constexpr	Pipe::PIPE_WIDTH - stała przynależna do klasy
std::vector<Pipe*>	dynamiczna tablica wskaźników na rury
new / delete	reczne zarządzanie pamięcią na stercie
static_cast<int>	bezpieczna konwersja float -> int przy rysowaniu
Q_OBJECT / connect	system sygnałów i slotów Qt
QTimer (16ms)	petla gry ~62.5 FPS
QPainter	rysowanie bez zewnętrznych assetów (plików PNG)
QRect::intersects	detekcja kolizji AABB
Override keyword	bezpieczne nadpisanie metod wirtualnych
Lista inicjalizacyjna	wydajna inicjalizacja pól w konstruktorze

Olaf Rysiewicz | Dawid Skatula | Szymon Panek
Programowanie II - C++ obiektowy | Politechnika Śląska 2026